

Autonomous Line Follower Robot

Hardware & Software Project Documentation

1. Project Overview

A Line Follower Robot is an automated guided vehicle that follows a visual line painted or embedded on the floor or ceiling. This project demonstrates a fundamental concept in robotics and automation. The robot uses Infrared (IR) sensors to distinguish between a black line and a white surface, processing the feedback through an Arduino microcontroller to actuate the drive motors accordingly.

Primary Objectives:

- To build a functional autonomous robot using an open-source hardware platform.
- To implement a closed-loop control system utilizing sensor feedback.
- To understand motor driver interfacing and digital logic in embedded systems.

2. Hardware Components Required

Component	Quantity	Specification / Purpose
Arduino UNO	1	Microcontroller board (ATmega328P) acting as the brain of the robot.
L298N Motor Driver	1	Dual H-Bridge driver to supply required current and direction to motors.
IR Sensor Module	2	Infrared transmitter-receiver pair to detect the line (Left and Right).
BO Motors	2	Geared DC motors for driving the wheels.
Wheels & Caster Wheel	2 & 1	Drive wheels attached to motors; a caster wheel for front balance.
Battery Power Supply	1	9V or 12V Li-ion battery pack to power the motor driver and Arduino.
Robot Chassis & Jumper Wires	1 Set	Acrylic/wooden base for mounting and wires for electrical connections.

Note on Component Selection: The L298N motor driver is essential because the Arduino's GPIO pins can only provide about 20-40mA of current, which is insufficient to drive DC motors. The L298N safely handles higher loads.

3. Working Principle & Logic

The system operates on the principle of light reflection. The IR sensor emits infrared light. When the light hits a white surface, it reflects back to the photodiode receiver. When it hits a black surface (the line), the light is absorbed, and no reflection occurs.

- **White Surface:** Light is reflected -> Sensor output is LOW (0).
- **Black Line:** Light is absorbed -> Sensor output is HIGH (1).

Movement Logic Truth Table

Left Sensor	Right Sensor	Robot Position Status	Motor Action (Left / Right)	Robot Movement
LOW (White)	LOW (White)	Robot is centered on the line	Forward / Forward	Move Straight
HIGH (Black)	LOW (White)	Drifting Right (Line is on the left)	Stop (or Reverse) / Forward	Turn Left
LOW (White)	HIGH (Black)	Drifting Left (Line is on the right)	Forward / Stop (or Reverse)	Turn Right
HIGH (Black)	HIGH (Black)	Perpendicular line or End of path	Stop / Stop	Stop Processing

4. Circuit Connections & Pin Mapping

The wiring logic ensures power is properly distributed. The Arduino logic pins control the motor driver, and the sensors feed data into the Arduino.

Module	Module Pin	Connected To (Arduino / Power)
L298N Motor Driver	12V Power In	Positive (+) terminal of 9V/12V Battery
	GND	Battery Negative (-) AND Arduino GND (Common Ground)
	5V Out	Arduino VIN (Powers the Arduino)
	IN1, IN2, IN3, IN4	Arduino Digital Pins 8, 9, 10, 11 (respectively)
Left IR Sensor	VCC	Arduino 5V
	GND	Arduino GND

5. Software Implementation (C++ Code)

The following code handles the logic execution. It reads the digital states of the sensors and applies the corresponding outputs to the motor driver pins based on the truth table defined earlier.

Part 1: Setup and Main Loop

```
// --- Pin Definitions --- const int in1 = 8; // Right Motor Forward const
int in2 = 9; // Right Motor Backward const int in3 = 10; // Left Motor Backward
const int in4 = 11; // Left Motor Forward const int leftSensor = 2; // Left IR
sensor const int rightSensor = 3; // Right IR sensor void setup() { // Configure
motor driver pins as OUTPUT pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
pinMode(in3, OUTPUT); pinMode(in4, OUTPUT); // Configure IR sensor pins as INPUT
pinMode(leftSensor, INPUT); pinMode(rightSensor, INPUT); } void loop() { // Read
digital logic from sensors int leftValue = digitalRead(leftSensor); int rightValue
= digitalRead(rightSensor); // LOGIC: Assuming Black Line = HIGH (1), White
Surface = LOW (0) // Condition 1: Go Straight if (leftValue == LOW && rightValue
== LOW) { moveForward(); } // Condition 2: Turn Left else if (leftValue == HIGH &&
rightValue == LOW) { turnLeft(); } // Condition 3: Turn Right else if (leftValue
== LOW && rightValue == HIGH) { turnRight(); } // Condition 4: Stop at
intersection else if (leftValue == HIGH && rightValue == HIGH) { stopRobot(); } }
```

Part 2: Movement Subroutines

```
// --- Movement Functions --- void moveForward() { digitalWrite(in1, HIGH);  
digitalWrite(in2, LOW); digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void  
turnLeft() { digitalWrite(in1, HIGH); digitalWrite(in2, LOW); // Stop left wheel  
to pivot left digitalWrite(in3, LOW); digitalWrite(in4, LOW); } void turnRight() {  
// Stop right wheel to pivot right digitalWrite(in1, LOW); digitalWrite(in2, LOW);  
digitalWrite(in3, LOW); digitalWrite(in4, HIGH); } void stopRobot() { // Bring all  
logic to LOW to stop motors digitalWrite(in1, LOW); digitalWrite(in2, LOW);  
digitalWrite(in3, LOW); digitalWrite(in4, LOW); }
```

6. Calibration and Troubleshooting

- **Motor Direction Reversal:** If the robot moves backward when it is supposed to move forward, the motor polarity is reversed. Do not change the code; instead, simply swap the two wires of that specific motor connected to the L298N output block.
- **Sensor Sensitivity:** If the robot fails to detect the black line, or detects the white surface as black, adjust the small potentiometer (screw) on the back of the IR sensor using a small screwdriver until the onboard indicator LED responds correctly to the surface change.
- **Sensor Placement:** Ensure the IR sensors are positioned close to the ground (about 5mm to 10mm clearance) but not touching it. They should be spaced apart slightly wider than the thickness of the black line.

Future Enhancements: To make the robot smoother, PWM (Pulse Width Modulation) can be introduced via the ENA and ENB pins on the L298N. This allows speed control, enabling PID (Proportional-Integral-Derivative) algorithms for high-speed, jitter-free line following.